Assessment Proforma 2025-26

Key Information

Module Code	CM2307
Module Title	Object Orientation, Algorithms and Data Structures
Module Leader	Ramalakshmi Vaidhiyanathan
Module Moderator	Dr Louise Wright
Assessment Title	Resit - Implementation using OOAD and DSA
Assessment Number	2
Assessment Weighting	60%
Assessment Limits	Max 2500 words for report

17th August - Try before 14th

This assessment's due date and time is specified on Learning Central. To view it click on the assessment's link, which will open *'Details & Information'* section for this assessment. Alternatively, go to *'Calendar'* section of a module, and select *'Due Dates'* option. You can change the view to daily or monthly, as well as use arrows to navigate the calendar.

The COMSC Assessment Calendar for all assessments is also available on Learning Central: by going to 'COMSC-SCHOOL' organisation -> **'Assessment & Feedback 2025 - 26'** folder -> **'COMSC Assessment Calendar 2025 - 26'** page.

Learning Outcomes

The learning outcomes for this assessment are as follows:

1. Appreciate the main features that are needed in a programming language to support the development of reliable, portable software and how key Object Oriented (OO) principles relate to those features.

2. Apply principles of good OO software design to the creation of robust, elegant, maintainable code.

4. Explain the difference between various fundamental algorithmic techniques and employ these for simple algorithm design.

6. Identify the most appropriate data structure and algorithm to solve a particular problem.

7. Explain and apply a range of design patterns.

Submission Instructions

All assessment files should be submitted in '*Assessment & Feedback'* folder on Learning Central.

Before you submit your assessment, you **must complete and submit** *'Assessment Coversheet'* form. Further instructions are given in this module's 'Assessment & Feedback' folder.

Your submission for this assessment should contain the following file(s):

Description	Compulsory or Optional?	**Type**	**Name**
Coursework Coversheet - Individual (you can find this in the COMSC-SCHOOL org)	Compulsory	One DOC (.docx) file	Coversheet.docx
Coursework Document	Compulsory	One DOC (.docx) file	CM2307_CW_StudentNumber.docx

You should only use the provided document template (CM2307_Coursework_Template.docx) as your coursework document, failing which marks will be deducted for non-compliance.

The detail of each sub-deliverable is provided in the individual task description, which is part of the Assessment Description section.

Note:

It is **your responsibility** to ensure that the correct document is submitted and that the submission is successful. It is also **your responsibility to** ensure proper access is made available to the marking team to both your Git code repo, failing which those components will accrue **zero marks.** Any changes to the submission (be it the document, code, or video) **after the due date will not be accepted.**

If you are unable to submit your work, email your submission to the module leader and comsc-submissions@cardiff.ac.uk. If the technical issue is with Cardiff University systems, also report it to the University IT Support: email it-support@cardiff.ac.uk, tel. +44 (0)29 2251 1111.

Submission Checklist:

Ensure you check the following before you submit your document on the portal.

Git Repo has all the committed code.	Yes/No
The Marking Team has been provided access to the Git Repo.	Yes/No
The submission template has been used to record the response to the assessment questions.	Yes/No
The submission document is saved as DOCX file and ready for submission.	Yes/No
All the necessary entries (student number, links to the repo etc.) are provided in the submission document.	Yes/No

Assessment Description

This assessment is split into two specific parts.

a) Part 1 expects you to refactor a given application implementation using Object Oriented design principles, design patterns, and produce design artifacts and refactored software application using Java.
b) Part 2 expects you to refactor a given application implementation focusing on the data structure and algorithms aspect of the application and to make it scalable, efficient and identify the complexity of your refactored code. You need to also produce the code.

Files provided along with the coursework proforma are:

a) StudentJobPortal.java
b) OptionalModule.java
c) ModuleCatalogue.java
d) CM2307_Resit_Coursework_Template.docx

These will be provided as a zip file to ensure all documents are downloaded without fail.

AI usage in this assessment:

AI can be used in the assessment for brainstorming and generating ideas for improving work. AI can also be used to make improvements to the clarity or quality of your work to improve the final output, but **no new content can be created using AI**. Use AI to create the structure but not in the final submission. You will have to declare the use of AI in your coursework template document provided. **Please note that the use of AI is optional. If you do not want to use AI for ethical, sustainable, or personal reasons, you do not have to use it. Please declare the same in the coursework template.**

Part 1 – 60%

Problem statement:

You are given starter code for a Student Job Portal system implemented in a procedural style with comments indicating intended functionality. The code works at a basic level, but it is difficult to extend, test, and maintain. Your task is to refactor the system into an object-oriented design that better separates responsibilities and supports future change.

The StudentPortal.java (used for this part) is provided along with the course proforma.

Your Task:

a) You must introduce suitable classes, use object-oriented principles effectively, and apply **at least TWO (2)** GoF (Gang of Four) design patterns in a justified way. You should not use design patterns from the same category (Creational, Behavioural, Structural) if the number of design patterns used is less than or equal to three.
b) You must use Java (version 17 and above). You should merge your code in your Git (**git.cardiff.ac.uk**) repo. Provide **Maintainer** access to your Git repo to the marking

team listed below. Do not use any external libraries for your application. You should use only the core Java libraries[1] (shouldn't require any additional downloads or jar files added to the path). You can use command line interface for any user interaction. ~~You can design for extending it to use GUI later, but it will not add more marks to your implementation.~~ Follow proper coding guidelines[2] and document your code. Your code will be reviewed for correctness, use of proper guidelines, use of design patterns you have reflected on, and if it matches the UML design you have provided.

c) Provide the design artefacts of your refactored design and implementation and add them to the coursework template provided (along with this proforma). You can use any UML design tool (draw.io, Visual Paradigm etc.) for your design. Ensure that your UML diagrams adhere to UML 2.0+ standards. Your design should include the following

not sure what this is, needs looking in to

   i. Use case diagram(s)
   ii. Class diagram (s)
   iii. Sequence diagram or activity diagram (s)

d) Compose a reflection in the coursework template document under Evidence and Evaluation marked as Part 1. Using evidence, evaluate how you have applied correct object-oriented design principles in your refactoring by comparing it to the principles that were broken in the original code. Provide a clear explanation of which design patterns were implemented, why they are appropriate and how they improve the design of your refactored system. **Provide clear evidence (in the form of code snippets or diagrams).** Code snippets and diagrams do not count towards final word count.
Word count for this reflection: 1500 words max.

Note: Do not overdesign this system. Use it to evidence your learning of the topics (OO design principles, and GoF design patterns).

Part 2 – 40%

Problem statement:

You are given a system for searching optional undergraduate modules offered in a particular academic year. This code (OptionalModule.java and ModuleCatalogue.java) follows decent object-oriented design principles, but it uses poor data structures for repeated searching and filtering, causing inefficient performance as the dataset grows.

Your Task:

We want O log N for this?
1 increment of time for each double of dataset

a) You must replace the inefficient structures with a more appropriate choice and redesign the search/filter logic so that the system performs significantly better in terms of their time and space complexity.

Need to get metrics of time in some form, look up command based method in MacOS

[1] [3] https://docs.oracle.com/en/java/javase/24/core/java-core-libraries1.html & https://openjdk.org/groups/core-libs/

[2] https://www.oracle.com/java/technologies/javase/codeconventions-contents.html

Need to make SAMPLE DATA
could use AI on a comma seperated list
for this?

b) You should add/edit class(es) to run the code with sample data to demonstrate your choice of data structure and how it helps in improving the efficiency of the data structure and the algorithms used.
c) You must use Java (version 17 and above). You should merge your code in your Git (**git.cardiff.ac.uk**) repo. Provide **Maintainer** access to your Git repo to the marking team listed below. Do not use any external libraries for your application. You should use only the core Java libraries[3] (shouldn't require any additional downloads or jar files added to the path). You can use command line interface for any user interaction. ~~You can design for extending it to use GUI later, but it will not add more marks to your implementation~~. Follow proper coding guidelines[4] and document your code.
d) Compose a reflection in the coursework template document under Evidence and Evaluation marked as Part 2. With evidence, justify the improved design of data structures and algorithms in the object-oriented context. Compare the original and refactored approaches using appropriate Big-O notation, demonstrating the improvement in time complexity. **Provide clear evidence (in the form of code snippets or diagrams).** Code snippets and diagrams do not count towards final word count.
Word count for this reflection: 1000 words max.

Marking Team:

1. Ramalakshmi Vaidhiyanathan (VaidhiyanathanR@cardiff.ac.uk)
2. Surya Thottam Valappil (ThottamValappilS@cardiff.ac.uk)
3. Aisha Gul (GulA3@cardiff.ac.uk)
4. Louise Wright (WrightL11@cardiff.ac.uk)

Penalties:

These are already mentioned in the Submission Instructions but reiterating them again for more clarity.

1. If the code is not provided in git.cardiff.ac.uk as mentioned above and/or proper access is not provided to the marking team, it will not be marked and the component associated with the code will be awarded **ZERO** (0).
2. If the reflections are not submitted in the provided coursework template document, a **penalty of 10%** will be added to the final marks.
3. If incorrect coursework document was submitted, you will be awarded **ZERO** (0) for the whole coursework. It is **your responsibility** to ensure that the correct document is submitted.

[3] [3] https://docs.oracle.com/en/java/javase/24/core/java-core-libraries1.html & https://openjdk.org/groups/core-libs/

[4] https://www.oracle.com/java/technologies/javase/codeconventions-contents.html

4. If you are submitting through multiple attempts, only the **latest attempt** will be **marked**. If the coursework template document is not present in that attempt, it will not be marked, and you will be awarded **ZERO (0)** for the whole coursework.

Assessment Criteria

Assessment Criteria:

The credits awarded will be based on the criteria given below.

Criteria	High First (80-100)	First (70-79)	2:1 (60-69)	2:2 (50-59)	3rd (40-49)	Marginal Fail (30-39)	Fail (0-29)
Code Quality (through Git repo code review) **30%** **(15% for task 1, 15% for task 2)**	Code shows highly modular design, adheres strictly to Java coding standards, ensures comprehensive error handling, has thorough documentation and comments where necessary, and implements well-structured OOP principles.	Code shows good modularity, mostly follows Java conventions, handles common errors, adequate comments and documentation, and uses OOP principles for implementation.	Code shows some reasonable structure, has few naming inconsistencies and doesn't fully follow coding guidelines, demonstrates limited error handling, very sparse documentation, and inconsistent use of OOP.	Code shows poor modularity, has frequent style violations, provides minimal error checking, lacks documentation, and has weak or incorrect use of OOP concepts.	Code shows very poor organization, violates most Java guidelines, has little to no error handling, provides no meaningful comments, and OOP absent or misunderstood.	Code has unreadable structure, looks non-functional, has no documentation or error handling.	Code is missing, unreadable or entirely non-functional.

Design Artefacts **20%** **(for task 1)**	Complete, accurate UML set: clear use-case, class, and sequence or activity diagrams provided; correctly labelled, consistent notation, reflects implementation decisions.	All three types of diagram present; minor inaccuracies or omissions; overall coherent and aligned with application structure.	At least two diagram types fully correct; the third partially present or contains errors; conveys main design ideas	Two diagrams supplied but with substantial omissions/ errors; third missing; design intent partially unclear.	One diagram is mostly correct; others are either missing or incorrect; limited insight into design.	Artefacts minimal or largely incorrect; fails to communicate design.	No design artefacts submitted.
Evidence and Evaluation 50% **(25% for task1, 25% for task 2)**	Refactoring is elegant, highly coherent, and strongly aligned with design principles. Pattern choices are excellent and very well integrated. Demonstrates exceptional understanding of object-	Refactoring is well structured and appropriate, with effective use of two or more GoF patterns. Very strong understanding of OO principles,	Refactoring improves the original code substantially and applies patterns reasonably well. Good understanding of core concepts with only minor inaccuracies. Good explanation of data	Refactoring is partially successful, but some design choices are weak or inconsistently applied. Adequate understanding of OO principles	Refactoring addresses the task only in a limited way, with partial use of OO principles and minimal pattern use. Basic understanding is demonstrated, but with notable gaps or misconceptions.	Refactoring is incomplete or poorly targeted, with weak object-oriented structure. Partial understanding, but important concepts are missing or incorrect. Weak or superficial justification, with little meaningful	Refactoring is largely absent, inappropriate, or incorrect. Very limited understanding of the task. Little or no justification, and no credible analysis of design or complexity.

	oriented design, GoF patterns, and data structure selection. Insightful evaluation, excellent Big-O analysis, strong justification of design trade-offs, and clear evidence of critical reflection.	patterns, and complexity. Clear justification of design decisions and sound complexity analysis with minor omissions only.	structure and algorithm choices and complexity, though analysis may lack depth or precision.	and algorithmic efficiency. Some justification is present, but discussion of complexity is limited or uneven.	Limited explanation of data structure and algorithm choices and basic complexity awareness only.	complexity analysis.	

Help and Support

Questions about the assessment can be asked to the module leader via e-Mail, or via Microsoft Teams. Responses will be provided using the same platform. We will aim to respond to questions within two working days (responses may take longer than two working days during holidays).

Feedback

Feedback on your coursework will address the above assessment criteria. Feedback and marks will be returned via Learning Central in accordance with the university's Academic Feedback Policy. You can see feedback return dates for each assessment on Learning Central: see **'COMSC Assessment Calendar 2025 - 26'** page.